Bounds & Generic Classes

CS300: Programming II

Cole Nelson

Learning Objectives

1. Be able to **use** `extends` to bound a type parameter.
2. Be able to **write** a generic class.
3. Be able to **use** a generic class with a type argument, like `Playlist<Song>`.

Oops!

I forgot to show you `getClass()` and `.class`.

```
Track t = new Song("Mr. Brightside", "The Killers", 222, "Hot Fuss");

System.out.println(t.getClass());   // class Song: asks the object
System.out.println(Song.class);     // class Song: names a class in code
```

`t` is declared a `Track`, but its object is a **`Song`**.

`getClass()` vs. `instanceof`

```
Track t = new Song("Mr. Brightside", "The Killers", 222, "Hot Fuss");

t instanceof Song                 // true
t instanceof Track                // true: a Song is a kind of Track
t.getClass() == Song.class        // true
t.getClass() == Track.class       // false: its class is exactly Song
```

`instanceof` asks: *is it a kind of...?* `getClass()` asks: *exactly which class?*

Recap: a `T` could be anything

```
public static <T> int totalSeconds(ArrayList<T> tracks) {
  int total = 0;
  for (T t : tracks) {
    total += t.getSeconds();   // undefined for the type T
  }
  return total;
}
```

A plain `T` only gets `Object`'s methods. Can we promise it's a `Track`? **Yes!**

Bounds

```
public static <T extends Track> int totalSeconds(ArrayList<T> tracks) {
  int total = 0;
  for (T t : tracks) {
    total += t.getSeconds();   // compiles: every Track has one
  }
  return total;
}
```

`T extends Track`: `T` **must be a** `Track`**,**
or a kind of `Track`**.**

Bounds

```
public static <T extends Track> int totalSeconds(ArrayList<T> tracks)

ArrayList<Song> library = new ArrayList<Song>();
ArrayList<String> artists = new ArrayList<String>();

totalSeconds(library);   // T is Song, a kind of Track
totalSeconds(artists);   // not applicable for the arguments (ArrayList<String>)
```

`T extends Track`: `T` **must be a** `Track`**,**
or a kind of `Track`**.**

Bound by an interface

```
public static <T extends Comparable<T>> T min(ArrayList<T> list)

Song first = min(library);     // T is Song, so a Song comes back
String a = min(artists);       // T is String
```

`Comparable` works as a bound, too. Still `extends`, never `implements`.

Eclipse Exercise

Get today's starter from Canvas. **Red already?**

TODO 1a. Bound `T` so `totalSeconds` compiles.

TODO 1b. One `min` for songs and podcasts.

Monday's `Playlist` gives back a `Track`

```
Playlist roadTrip = new Playlist("Road Trip", 10);
Playlist commute = new Playlist("Commute", 5);

Song first = (Song) roadTrip.get(0);        // a cast...
Podcast next = (Podcast) commute.get(0);    // ...every time

first.getAlbum();      // only a Song has one
next.getEpisode();     // only a Podcast has one
```

Every `get` needs a cast. **Not ideal.**

A generic class

```
public class Playlist<T extends Track> {
  private String name;
  private int size;

  public void add(T track) { ... }
  public T get(int index) { ... }
}
```

`<T>` goes **after the class name**. Then fields, parameters, and return types can all use it.

Red already?

We may also consider changing `tracks` to be...

```
this.tracks = new T[capacity];
```

... but we would encounter ...

```
Cannot create a generic array of T
```

Java won't make an array of `T`. **Now what?**

Keep Monday's `Track[]`

```
private Track[] tracks;   // every T is a Track, so it fits...

public T get(int index) {
  return (T) this.tracks[index];   // ...and we hand back a T
}
```

Eclipse warns: *Unchecked cast from Track to T*. It's safe: `add` only lets a `T` in.

No bound? Then a `T` might not be a `Track`, so use an `Object[]`.

What else can't a `T` do?

```
T track = new T();          // Cannot instantiate the type T
T[] tracks = new T[10];     // Cannot create a generic array of T
if (x instanceof T)         // won't compile either
```

Same reason every time: while the program runs, a `Playlist` doesn't know what its `T` is.

Tophat Q1

A quick check on what a generic class can do with `T`.

→ Open Tophat to vote.

Pick a type

```
Playlist<Song> roadTrip = new Playlist<Song>("Road Trip", 10);
Playlist<Podcast> commute = new Playlist<Podcast>("Commute", 5);

Song first = roadTrip.get(0);       // a Song back: no cast!
Podcast next = commute.get(0);      // a Podcast back: no cast!

first.getAlbum();
next.getEpisode();
```

Same getters **without the casts**.

Eclipse Exercise

TODO 2a. Make `Playlist` generic.

TODO 2b. Create playlists for a road trip (songs) and a commute (podcasts).

Tophat Q2

Does the last line compile?

```
Playlist<Track> mix = new Playlist<Track>("Mix", 5);
mix.add(brightside);   // brightside is a Song
Song s = mix.get(0);
```

→ Open Tophat to vote. *Then* we try it in Eclipse.

Abstract data types

Definition: An **abstract data type** (ADT) is described by *what* you can do with it, not *how* it's built.

Our `Playlist` is a **list**: add, get, and size, by position. We built ours on an array.

ADTs in a music app

ADT	Rule	In our app
List	in order, by position	a playlist
Stack	last in, first out	the Back button
Queue	first in, first out	Up Next
Set	no repeats	Liked Songs
Map	look up a value by its key	plays per song

Java writes ADTs as interfaces

```
public interface List<E> {   // a few of its methods
  boolean add(E e);
  E get(int index);
  int size();
}

List<Song> songs = new ArrayList<Song>();
```

`List` is a **generic interface**: the *what*. `ArrayList` is one *how*, and `LinkedList` is another.

Questions?